

This lecture introduces and explains lambda functions in Python. Here's a breakdown:

- What are Lambda Functions?
 - Lambda functions are anonymous functions (functions without a name).
 - They provide a concise way to create small, single-expression functions.
 - They can be used as an alternative to regular functions, especially for simple operations.
- Syntax and Structure:
 - The basic syntax is `lambda arguments: expression`.
 - `lambda` is a keyword that indicates the creation of a lambda function.
 - `arguments` are the input parameters the function accepts (e.g., `a, b`).
 - `expression` is the single expression that the function evaluates and returns.
- Example: Adding Two Numbers
- A regular function:

```
def add(a, b):
```

 - ```
 return a + b
```
  -
- The equivalent lambda function:

```
c = lambda a, b: a + b
```
- ```
result = c(4, 5) # result will be 9
```

 -
- How Lambda Functions Work:
 - The `lambda` keyword signifies a lambda function.
 - Input arguments are specified after `lambda` (e.g., `a, b`).
 - A colon `:` separates the arguments from the expression to be evaluated.
 - The expression's result is automatically returned.
- Use Cases and Best Practices:
 - Lambda functions are suitable for small, simple logic or single-line operations.
 - They are often used with functions like `map()`, `filter()`, and `sorted()`.
 - For complex logic or multi-line operations, regular functions are generally preferred for better readability and maintainability.
 - If you are not comfortable with lambda functions, it's perfectly acceptable to use regular functions instead.
- Examples Demonstrated:
 - Squaring a Number:

```
square = lambda data: data**2
```
 - Finding the Maximum of Two Numbers:

```
max_val = lambda a, b: a if a > b else b
```
 - Checking if a Number is Even or Odd:

```
even_odd = lambda number: "even" if number % 2 == 0 else "odd"
```
- Key Takeaways:
 - Lambda functions offer a concise way to define simple functions inline.
 - They are best suited for short, single-expression operations.
 - Readability and maintainability should be considered when choosing between lambda and regular functions.
 - Regular functions are better for complex logic.
 -